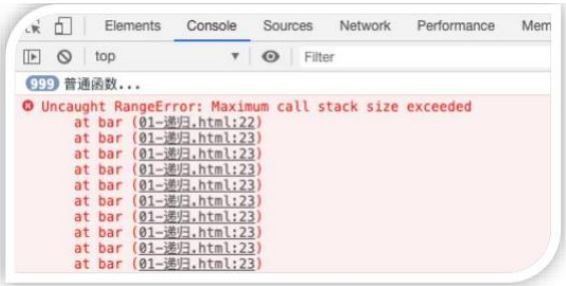
- **递归函数的介绍**
  - 如果一个函数内部调用自已本身，并且通过条件控制避免陷入死循环，那么我们称这个函数为递归函数。
  - 基本使用

```
function foo() {
    console.log('普通函数...');
    foo(); // 函数内部调用自己
}
// 调用函数
foo();
```



    - 注意：上述代码会导致 foo 一直重复被调用， 会陷入死循环，浏览器会以报错形式终止执行
  - 递归要加结束条件

```
let i = 1;
function bar() {
    i++; // 自增 1
    // 通过条控制是否调用自己
    if (i <= 10) {
        bar();
    }
}
// 调用函数
bar();
```

- **递归函数的特性**
  - 递归函数有 3 个特征：
    - 重复执行
    - 调用自身
    - 【必须】要有条件控制，避免死循环

```
// 利用递归求 1~ 100 的和
let s = 0, i = 0;
function foo() {
    // 条件控制避免陷入死循环
    if (i >= 100) return;
    // 正常的求和运算
    i++;
    s += 1;
    foo(); // 自调用
}
foo();
```

- 总结：递归本身是一种循环操作，简单情况下可以替换循环语句的使用。

- **节点操作——插入节点**
  - 即创造出一个新的网页元素，再添加到网页内，一般先创建节点，然后插入节点
  - 创建元素节点方法：

```
// 创造一个新的元素节点
document.createElement('标签名')
```

```
// 克隆一个已有的元素节点
元素.cloneNode(布尔值)
```

    - cloneNode会克隆出一个跟原标签一样的元素，括号内传入布尔值
      - 若为true，则代表克隆时会包含后代节点一起克隆
      - 若为false，则代表克隆时不包含后代节点
      - 默认为false
  - 要想在界面看到，还得插入到某个父元素中

- 插入到父元素的最后一个子元素

```
// 插入到这个父元素的最后
父元素.appendChild(要插入的元素)
```

- 插入到父元素中某个子元素的前面

```
// 插入到某个子元素的前面
父元素.insertBefore(要插入的元素, 在哪个元素前面)
```

- **节点操作——删除节点**
  - 若一个节点在页面中已不需要时，可以删除它
  - 在 JavaScript 原生DOM操作中，要删除元素必须通过父元素删除
  - 语法

```
父元素.removeChild(要删除的元素)
```

    - 注：如不存在父子关系则删除不成功

- **节点操作——替换节点**
    - 替换节点是指把一个已经存在页面中的元素替换成新的元素
    - 方法一语法：

```
父元素.replaceChild(新节点, 被替换节点)
```

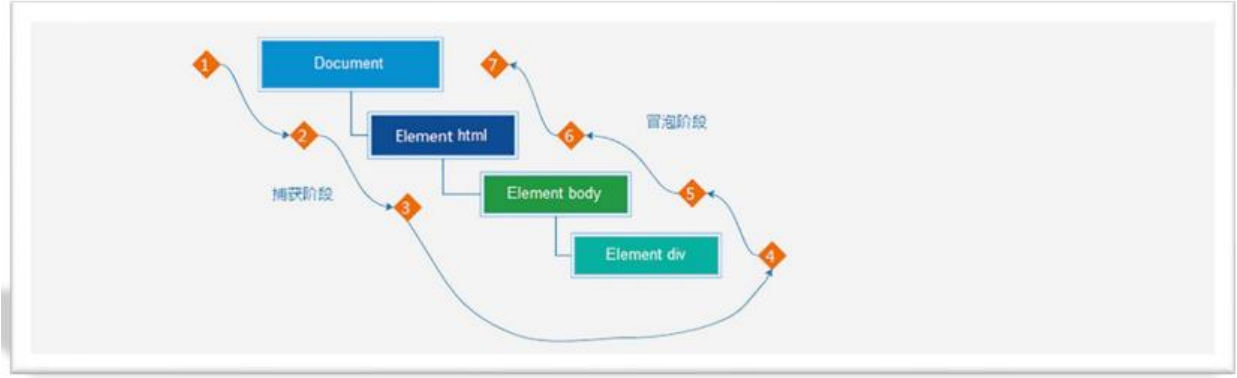
    - 方法二语法：

```
父元素.innerHTML = 内容
```

    - 方法二的innerHTML本质上是修改双标签里面的内容，只不过遇到标签也会解析成元素

- **节点操作——查找节点**
    - 父子关系查找：
        - 获取父节点方法
            - parentNode 属性 返回最近一级的父节点 找不到返回为null
        - 获取子节点
            - childNodes
                - 获得所有子节点、包括文本节点（空格、换行）、注释节点等
            - children
                - 仅获得所有元素节点
        - 查找兄弟的属性
            - previousSibling
                - 找到上一个兄弟节点，包含文本、注释等节点
            - prevuisElementSibling
                - 找到上一个兄弟元素，只找到元素
            - nextSibling
                - 找到下一个兄弟节点，包含文本、注释等节点
            - nextElementSibling
                - 找到下一个兄弟元素，只找到元素
        - 查找节点名称
            - nodeName
                - 返回大写的标签名

- **事件流和两个阶段说明**
    - 事件流指的是事件完整执行过程中的流动路径



- 说明：假设页面里有个div，当触发事件时，会经历两个阶段，分别是捕获阶段、冒泡阶段
- 简单来说：捕获阶段是 从父到子 冒泡阶段是从子到父

- **事件捕获和事件冒泡**
  - 事件冒泡是默认存在的
  - 它指的是：当一个元素触发事件后，会依次向上调用所有父级元素的同名事件
  - 事件捕获需要写对应代码才能看到效果
  - 代码：

```
元素.addEventListener(事件名, 回调函数, true)
```

  - 说明：
    - addEventListener第三个参数传入true代表是捕获阶段触发（很少使用）
    - 若传入false代表冒泡阶段触发，默认就是false
    - 若是用 L0 事件监听，则只有冒泡阶段，没有捕获

- **阻止事件流动**
  - 因为默认就有冒泡模式的存在，所以容易导致事件影响到父级元素
  - 若想把事件就限制在当前元素内，就需要阻止事件流动
  - 阻止事件流动需要拿到事件对象
  - 语法：

```
事件对象.stopPropagation()
```

  - 此方法可以阻断事件流动传播，不光在冒泡阶段有效，捕获阶段也有效
  - 鼠标经过事件：
    - mouseover 和 mouseout 会有冒泡效果
    - mouseenter 和 mouseleave 没有冒泡效果(推荐)

- **事件委托**
  - 事件委托是利用事件流的特征解决一些开发需求的知识技巧
  - 过程：

    - 把事件注册给上级元素
    - 用e.target找到目标元素
    - 用nodeName判断是否是我们要找的元素
  - 总结：
    - 事件委托其实是利用事件冒泡的特点，给父级元素加事件
    - 事件对象.target 可以获得真正触发事件的元素